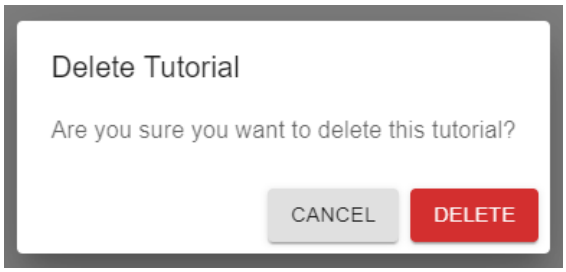
Lab4b Retrieve and Edit Data

Learning Outcomes

- Add new page to edit tutorial and retrieve tutorial by id
- Add function to update tutorial
- Add function to delete tutorial



Activity 1: Add new page to edit tutorial and retrieve tutorial by id

1. Use VS Code to open the folder **learning-app**.

2. Open 2 terminals, one for **server**, the other for **client**. Start the server and the client.

3. Under the folder client/src/pages, add a new file EditTutorial.jsx

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

4. In EditTutorial.jsx, type **rfce** to generate the starting code:

```
import React from 'react'

function EditTutorial() {
  return (
    <div>EditTutorial</div>
  )
}

export default EditTutorial
```

5. In App.jsx, add a new route to edit tutorial:

```
import EditTutorial from './pages/EditTutorial';
…
            <Route path={"/addtutorial"} element={<AddTutorial />} />
            <Route path={"/edittutorial/:id"} element={<EditTutorial />} />
```

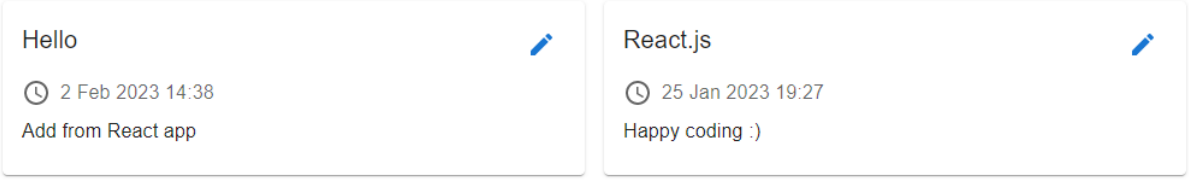
The parameter **id** is in the route path.

6. In Tutorials.jsx, add new box to include both title and link icon button to edit tutorial:

```
import { AccessTime, Search, Clear, Edit } from '@mui/icons-material';
…
<CardContent>
    <Typography variant="h6" sx={{ flexGrow: 1 }}>
        {tutorial.title}
    </Typography>
    <Box sx={{ display: 'flex', mb: 1 }}>
        <Typography variant="h6" sx={{ flexGrow: 1 }}>
            {tutorial.title}
        </Typography>
        <Link to={`/edittutorial/${tutorial.id}`}>
            <IconButton color="primary" sx={{ padding: '4px' }}>
                <Edit />
            </IconButton>
        </Link>
    </Box>
    …
</CardContent>
```


Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

7. Start the client. In the browser, click the edit icon to go to the edit tutorial page.



Observe that the tutorial id is in the url.

8. Go to VS Code, in EditTutorial.jsx, change code to retrieve tutorial by id:

```
import React, { useEffect } from 'react';
import { useParams } from 'react-router-dom';
import http from '../http';

function EditTutorial() {
    const { id } = useParams();

    useEffect(() => {
        http.get(`/tutorial/${id}`).then((res) => {
            console.log(res.data);
        });
    }, []);
```

The useParams hook returns an object of the dynamic params from the current URL.

9. In the browser console, observe that the tutorial object is logged.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

Activity 2: Add function to update tutorial

1. Go to VS Code, in EditTutorial.jsx, change code to display page title and form:

```
import { Box, Typography, TextField, Button } from '@mui/material';
…

function EditTutorial() {
…
    return (
        <Box>
            <Typography variant="h5" sx={{ my: 2 }}>
                Edit Tutorial
            </Typography>
            <Box component="form">
                <TextField
                    fullWidth margin="dense" autoComplete="off"
                    label="Title"
                    name="title"
                />
                <TextField
                    fullWidth margin="dense" autoComplete="off"
                    multiline minRows={2}
                    label="Description"
                    name="description"
                />
                <Box sx={{ mt: 2 }}>
                    <Button variant="contained" type="submit">
                        Update
                    </Button>
                </Box>
            </Box>
        </Box>
    );
}
```

The code is similar to what we have in AddTutorial.jsx.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

2. In the browser, observe that the form is displayed.

3. Edit code to use state for tutorial and loading:

```
import React, { useState, useEffect } from 'react';
…

    const { id } = useParams();

    const [tutorial, setTutorial] = useState({
        title: "",
        description: ""
    });
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        http.get(`/tutorial/${id}`).then((res) => {
            setTutorial(res.data);
            setLoading(false);
        });
    }, []);
```

The **tutorial** state variable is used to store the tutorial data. When the data is received from api response, the state value will be updated, which will cause a re-render of the component with the updated data.

The **loading** state variable is used to conditionally render content. When loading is true, you can display a loading indicator or placeholder content. When loading becomes false, it indicates that the data has been loaded, and you can render the actual tutorial.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

4. Use loading state as the condition to render the form:

```
{
    !loading && (
        <Box component="form">
        …
        </Box>
    )
}
```

5. Add code to use formik and yup:

```
import { useFormik } from 'formik';
import * as yup from 'yup';
…

    useEffect(() => {
    …
    }, []);

    const formik = useFormik({
        initialValues: tutorial,
        enableReinitialize: true,
        validationSchema: yup.object({
            title: yup.string().trim()
                .min(3, 'Title must be at least 3 characters')
                .max(100, 'Title must be at most 100 characters')
                .required('Title is required'),
            description: yup.string().trim()
                .min(3, 'Description must be at least 3 characters')
                .max(500, 'Description must be at most 500 characters')
                .required('Description is required')
        }),
        onSubmit: (data) => {
            data.title = data.title.trim();
            data.description = data.description.trim();
            http.put(`/tutorial/${id}`, data)
                .then((res) => {
                    console.log(res.data);
                });
        }
    });
```

The initial values of the formik are based on the **tutorial** state variable.

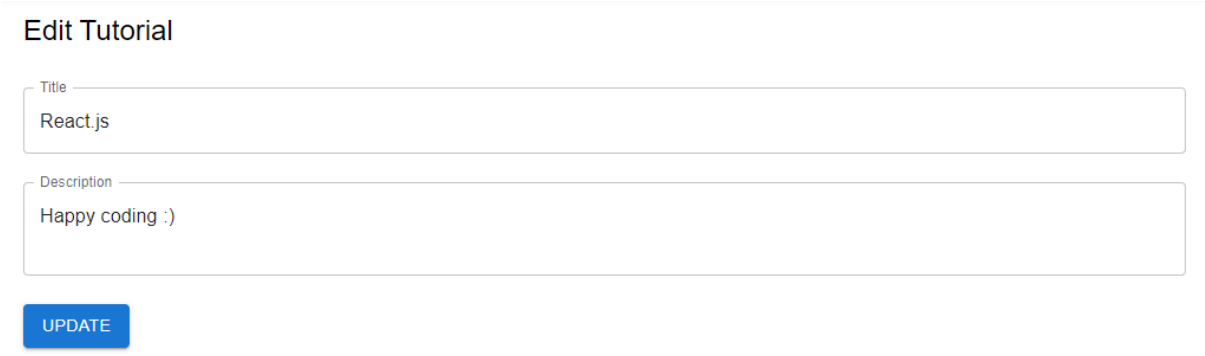
The formik parameter **enableReinitialize** is set to true, because the value of tutorial state will be changed after getting api response for the tutorial object.

The **formik** object will be used in the form.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

6. Refer to AddTutorial.jsx, try yourself to use formik object in the form.

7. In the browser, observe that the existing tutorial attributes are displayed:



8. Enter new title and description, click on the button Update. Observe that the updated tutorial object is logged in the browser console.

9. In the app bar, click the link Tutorials. Observe that the tutorial is updated in the list.

10. Go to VS Code, In EditTutorial.jsx, try yourself to navigate to the tutorials page after getting response from the put request.

11. In the browser, test the navigation.

Activity 3: Add function to delete tutorial

1. Go to VS Code, In EditTutorial.jsx, add function to delete tutorial:

```
const formik = useFormik({
…
});

const deleteTutorial = () => {
    http.delete(`/tutorial/${id}`)
        .then((res) => {
            console.log(res.data);
            navigate("/tutorials");
        });
}
```

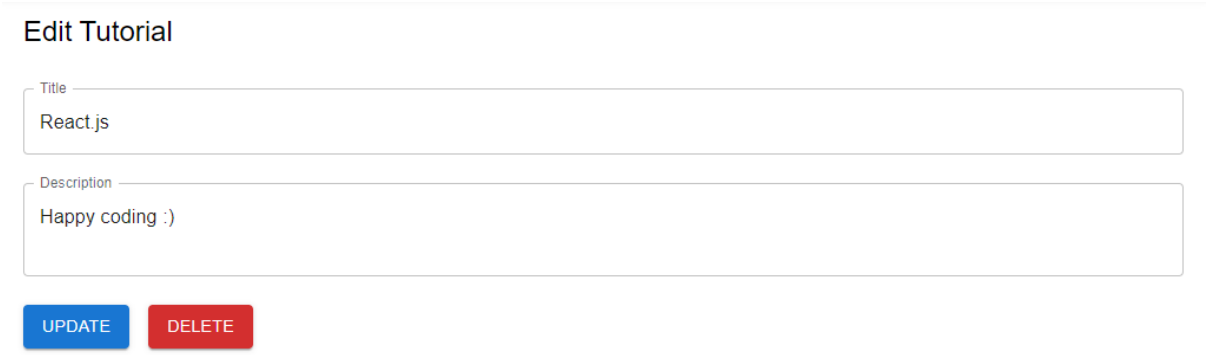
Call api to delete the tutorial and navigate to the tutorials page.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

2. Add delete button with the on click handler:

```
<Box sx={{ mt: 2 }}>
    <Button variant="contained" type="submit">
        Update
    </Button>
    <Button variant="contained" sx={{ ml: 2 }} color="error"
        onClick={deleteTutorial}>
        Delete
    </Button>
</Box>
```

3. In the browser, test delete function.



4. Go to VS Code, In EditTutorial.jsx, add new state open:

```
    const formik = useFormik({
    …
    });

    const [open, setOpen] = useState(false);

    const handleOpen = () => {
        setOpen(true);
    };

    const handleClose = () => {
        setOpen(false);
    };
```

The state open will be used for control dialog visibility for delete confirmation.
The initial value is false, meaning the dialog is hidden.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

5. Update on click handler for the delete button:

```
<Button variant="contained" sx={{ ml: 2 }} color="error"
    onClick={handleOpen}>
    Delete
</Button>
```

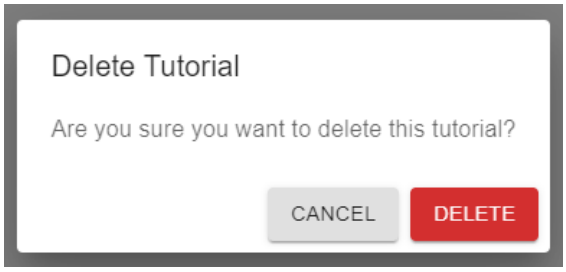
6. Add MUI dialog below the form for delete confirmation:

```
import { Dialog, DialogTitle, DialogContent, DialogContentText,
DialogActions } from '@mui/material';
…
{
    !loading && (
        <Box component="form">
        …
        </Box>
    )
}
…
<Dialog open={open} onClose={handleClose}>
    <DialogTitle>
        Delete Tutorial
    </DialogTitle>
    <DialogContent>
        <DialogContentText>
            Are you sure you want to delete this tutorial?
        </DialogContentText>
    </DialogContent>
    <DialogActions>
        <Button variant="contained" color="inherit"
            onClick={handleClose}>
            Cancel
        </Button>
        <Button variant="contained" color="error"
            onClick={deleteTutorial}>
            Delete
        </Button>
    </DialogActions>
</Dialog>
```

When clicking on the delete button besides the update button, open the dialog.
When clicking on the delete button inside the dialog, call the delete tutorial function.

Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.

7. In the browser, test the delete with confirmation.



Copyright © 2024. Singapore institute of technology and Nanyang Polytechnic. All rights reserved.